



PES UNIVERSITY, Bangalore

(Established under Karnataka Act No. 16 of 2013)

Department of Computer Science & Engineering

Session: Jan-May, 2026

Course Information (56 Hours)

UE23CS341B-COMPILER DESIGN (4-0-0-0-4)

Class #	Unit	Topic	Chapter Title /Reference Literature	% Coverage	
				Unit	Total
1	I	Introduction and Language Processing system	T1: Chapter 1: Introduction, 1.1 - 1.2 Chapter 2: 2.6, 2.7 Lexical Analysis, 3.1-3.5, 3.8 T1: Chapter 2 2.4 T1: Chapter 4 Syntax Analysis, 4.1.1, 4.1.3, 4.1.4, 4.3.3, 4.3.4, 4.4	25	25
2,3		Phases of a compiler.			
4		Lexical Analysis: The Role of the Lexical Analyzer			
		Input buffering			
5		Specification and Recognition of Tokens			
6		Design of a Lexical Analyzer Generator.			
7,8		Introduction to Lex tool			
PE1		Develop a complete Lexical Analyzer for the C programming language using LEX/FLEX. The Lexer should recognize all valid C tokens (keywords, identifiers, constants, string/character literals, operators, punctuators). Input: Any C program. Output: Stream of tokens (with their type and lexeme). - Tokens are printed to stdout in a readable format (e.g., TOKEN_TYPE: lexeme). Or - returned as a structured format (JSON, CSV, .txt etc.).			
9		Syntax Analysis: The role of the parser, Syntax Error Handling, Error-Recovery Strategies. Introduction to different parsers (Top Down and Bottom-up Parsers).			
10		Grammar enhancement techniques: - Eliminating ambiguity - Eliminating left-recursion - Left Factoring (Elimination of common prefixes)			
11		RDP without Backtracking, Construction of FIRST and FOLLOW sets			
12					
13		Top-down parsing: LL(1) grammar, Construction of predictive parsing (or LL(1)) table.			

14		Predictive parser (LL(1) Parser). Understanding LL(*)				
15		Error recovery in LL(1) Parser				
16	II	Bottom-up parsing: Handle and Handle pruning, Shift-Reduce Parsing, Conflicts During Shift-Reduce Parsing	T1: Chapter 4 Syntax Analysis, 4.5 - 4.7.4 & 4.9	25	50	
17		LR parsing: Items, LR(o) Automaton, Augmented grammar, Closure of Item Sets, and Function GOTO.				
18		Viable prefixes and LR-parsing algorithm				
19,20		LR(o) parsing				
21, 22		Simple-LR Parsing/SLR Parsing				
23		Canonical LR/CLR Parsing				
24		LALR Parsing Comparison of all Bottom-up parsers.	T1: Chapter 5 Syntax Directed Translation 5.1, 5.2.1–5.2.4,			
PE2		Introduction to YACC/Bison(Parser Generator)				
		Develop a YACC/Bison parser to validate the syntax of a simplified subset of the C language. The grammar must support: Multiple variable declarations with basic types (int, float, char, double) and comma-separated declarators. if, if-else, and do-while control statements with support for nested blocks { ... }. Arithmetic and relational expressions. Input: Any C-like source program conforming to the above subset (from stdin or a file). Output: On success: Syntax valid. On failure: Syntax error at line <n>, token <t>: <message> Assumptions: - Use a LEX/FLEX lexer to provide tokens (ID, NUM, IF, ELSE, DO, WHILE, operators, punctuation). - Ignore comments and whitespace in the lexer.				
		A1				Assignment 1: Extend PE2 to handle: for (eg: for(i=0, j=0; i<p&&j<q; i++, j++)) and while and switch statements. Variable declaration and initialization. (eg: int a=5, b, c, d=10;) Array declarations (eg: int a[15]; int a[10][10]; int a[1][2][3][4]; int a[4][4], b[5];)
						Syntax-Directed Translation (SDT): - Attribute grammar, Synthesized & Inherited Attributes. - L-attributed SDT and S-attributed SDT
	25 26		Syntax-Directed Definition (SDD): Evaluating an S-Attributed SDD of a simple desk calculator			
27	Evaluating an SDD over a given input: (1-example) Construct Parse Tree for given input.					

		- Construct Dependency graph. - Topologically sort the nodes of Dependency graph. - Produce as output Annotated Parse Tree.			
28		- Evaluating an L-Attributed SDD of a simple desk calculator. - L-Attributed SDD to update type and size of an variable in symbol table (L-Attributed SDD for simple declaration statement)			
29		Dependency graph for the - SDD of simple declaration statement: float x, y, z - SDD to find out whether the given type is a Basic Type or an array type and calculate the width required			
30		S-Attributed SDD Examples - SDD to convert Binary Number to Decimal - SDD to convert Binary Fraction to Decimal - SDD to count number of Balanced Parenthesis - SDD to convert a infix to postfix			
		ISA-1			
PE3		Develop a program (with a LEX/FLEX lexer and optional Bison parser) to implement a symbol table for a subset of the C language. The symbol table must record, for each identifier (variables, functions, parameters, typedef names, enum constants, struct/union tags): Name (lexeme) Kind (variable/function/parameter/typedef/enum-const/struct-tag/union-tag) Type (base type + declarator info: pointers, arrays, function signatures) Storage class (auto, static, extern, register) Size in bytes Scope (scope level, e.g., global/file/function/block) Definition location (line, column, file) and use-sites list (optional) Initializer value (if constant and computable at compile time; otherwise null) Input: C-like source code for the chosen subset (at minimum: declarations, function definitions, blocks). Output: A stream or table of symbol entries in a clear format (e.g., tabular)			
31		Applications of S-Attributed SDD: Construction of syntax tree for an expression and statements			
32		Parser-Stack implementation of postfix SDT's			

33 34	III	S-attributed SDD to S-attributed SDT(Postfix SDT) Conversion. L-attributed SDD to L-attributed SDT Conversion rules. L-attributed SDD's and SDT's to generate Intermediate code for - while statement - do-while statement - if-else statement - for statement - Conditional Boolean expressions	T1: Chapter 5 Syntax Directed Translation 5.3.1, 5.3.2, 5.4.1 – 5.4.3 , 5.5.4 T1: Chapters6 Intermediate –Code Generation ,6.1- 6.1.1, 6.2, 6.6: 6.6.1, 6.6.2 T1: Chapter 8 Code Generation, 8.4	25	75
PE4		Develop a YACC/Bison parser for Expression Evaluation: Given for example an input that contains statements such as, a=5, b=8, c=9, d=a+b*c. After arithmetic expression evaluation update the symbol table with the value of 77 for variable d. Check for variable not declared error, mismatch types in expression and display relevant error messages.			
35 36		Bottom-Up Parsing of L-Attributed SDD's for Intermediate code generation for - while statement - if statement			
37		Intermediate-Code Generation: Introduction Variants of Syntax Trees – Directed Acyclic Graphs(DAG's) for Expressions.			
PE5		Develop a YACC/Bison parser (with a LEX/FLEX lexer) to construct an Abstract Syntax Tree (AST) for arithmetic expressions and print the tree in postorder (Reverse Polish) traversal. Output: for each expression, print its postorder traversal (tokens separated by spaces).			
38 39 40 41		Three-Address Code – Addresses and Instructions. Quadruples, Triples and indirect triples Static Single-Assignment(SSA)			
42		Basic Blocks and Flow Graphs/Control Flow Graphs(CFGs)			
43		Next-Use Information/Algorithm			
PE6		Develop a YACC/Bison parser (with a LEX/FLEX lexer) to generate three-address Intermediate Code (IC) for arithmetic expressions and assignments, printing each instruction in quadruple format. Define a function to generate temporaries. Quadruple format: Print each IC instruction as (op, arg1, arg2, result)			
A2		Assignment 2: Extend PE5 (AST construction) and PE6 (ICG) for if-else and do-while statement. Use the following productions $S \rightarrow do \{S\} while (C); S \mid if (C) \{S\} else \{S\} S$ $C \rightarrow T_ID \text{ rel } T_ID$ $rel \rightarrow < \mid > \mid <= \mid >= \mid == \mid !=$			

44	IV	Machine Independent Optimization: Different Optimization techniques	T1: Chapter 8 Code Generation, 8.5 Chapter 9 Machine Independent Optimization : 9.1, 9.2.5 Chapter 7 Run Time Environment, 7.1–7.3 Chapter 8 Code Generation, 8.1– 8.3, 8.6 8.8.4	25	100
45,46		Optimization on CFG (Global Optimization).			
47,48		Live-variable Analysis.			
49		Code Generation: Issues in the design of a Code generator.			
50		Run-Time Environments: Introduction			
51		Storage Organization (Runtime storage)			
		Activation Tree & Activation Record			
		Calling Sequence and Return Sequence			
52		Variable-Length Data on the Stack			
53		Introduction to ML language and Access to Nonlocal Data on the Stack			
54,55		Access links and Displays Code Generator Algorithm			
56		Register Allocation by Graph Coloring			
Home work		Self-Study: Introduction to LLVM and Clang. Simple exercises			
		ISA-2			

Reference Textbooks:

Book Type	Code	Title & Author	Publication Information		
			Edition	Publisher	Year
Text Book	T1	Compilers–Principles, Techniques and Tools Alfred V. Aho, Monica S. Lam, RaviSethi, Jeffery D. Ullman	2 nd	Pearson Education	2023
Reference Book	R1	“Modern Compiler Design” , Dick Grune, Kees van Reeuwijk, Henri E. Bal, Ceriel J.H. Jacobs, KoenLangendoen,	2 nd	Pearson Education	2012

ISA-ESA Evaluation Policy:

	Details	Conducted for	Scaled down to	Total
ISA	ISA-1 (Units 1 & 2)	40	20	40
	ISA-2 (Units 3 & 4)	40	20	
Experiential Learning (EL)	6 - Programming Exercises (PEs) Banana Problem: PE1 and PE2 Orange Problem: PE3, PE4, PE5 and PE6	60	06	10
	Jackfruit Problem: Two Coding Assignments	40	04	
ESA		100	50	50
Total				100